

Restoring “Engineering” to Software Engineering

Marcelo d’Amorim, Wesley Assunção, Sarah Heckman, Sandeep Kuttal,

Tim Menzies, John-Paul Ore, Kathryn Stolee, Bowen Xu

NC State University, USA

August 4, 2026

{mdamori, wguezas, sarah_heckman, skuttal, tjmenzie, jwore, ktstolee, bxu22}@ncsu.edu

Abstract

We report on a faculty brainstorming session focused on updating the software engineering undergraduate curriculum for the age of AI. By mapping curriculum needs against the transition from ACM CC2005 to the proposed CS2023 guidelines, we identify critical gaps in student competencies. We propose a minimal-impact curriculum update: i.e., **two new subject areas** designed to address the economic, psychological, and rigorous verification requirements of AI-driven development.

1 Introduction

As AI generative tools reshape the technological landscape, Software Engineering (SE) is pulled in disparate directions. The field has become a vessel for whatever tool currently captures the “zeitgeist”.

To cure that vertigo, this document articulates a vision of SE in this new context: *what it is*, and *what it could be*. Our stance will be **radical enthusiasm** tempered by **critical reflection**. New technologies can contain bugs and require oversight (top part of Table 1). Additionally, the benefits of AI tools do not come for free; they involve several aspects that should be taken into account (middle part of Table 1). That said, we are excited to make extensive use of AI in our research and teaching (see Table 2). Accordingly, we recommend:

Principle 1: SE and AI at NC State

AI, carefully. AI enthusiasm plus engineering rigor.

Given this context, we argue that **we do not need classes on how to prompt; we need classes on how to govern**.

The remainder of this paper develops our vision of “AI, carefully” by grounding enthusiasm for AI generative tools in SE. Section 2 surveys recent *trends in Software Engineering*, highlighting shifts in developer activity, the growing centrality of testing, the rise of neurosymbolic systems, and emerging economic constraints in the AI marketplace. Section 3 reasserts the role of *SE as managed creation and maintenance discipline*, requiring quality under constraints, and discusses its implications for management, quality assurance, optimization, and neurosymbolic integration. Section 4 bridges this *vision to established educational frameworks*, mapping our proposals to Bloom’s Taxonomy, the ACM CS2023 curriculum, and ABET accreditation requirements. Section 5 argues that *SE remains indispensable* even in an era of AI-assisted development, and Section 6 discusses an action plan to *enrich the undergraduate experience in SE* with that in mind.

Table 1: AI tools require engineering oversight.

Issue	Evidence
Cognitive debt	With generative and agentic AI accelerating how software gets built, teams can move very fast while slowly losing shared understanding of why systems are designed the way they are and how they can be safely changed [90]
Hallucinations	52% of ChatGPT answers have incorrect info [38]
Human error	Humans overlook AI errors 39% of the time [38]
Code quality	70% faulty: 29% correct, 51% partial, 20% incorrect [107]
Expert caution	Google developers accept only 25% of AI suggestions cite
Training bias	LLMs regenerate bugs from training data [37]
Task failure	LLMs fail at technical debt tasks [55]
Additional Concerns from this document	
Money	Providing student access to closed models can be expensive
Energy	High consumption. “Green SE” a hard constraint
Explanation	Subsymbolics: no causal/ explanatory reasoning
Verification	Students can prompt but often cannot verify output
Planning	“Vibe coding” skips planning, increasing testing costs
Ownership	Copy-pasting reduces developer job satisfaction
Maintenance	Lack of “under-the-hood” skills → disasters?
Understanding	Hard to debug/edit auto-generated code
Concerns from the AI Marketplace	
Bubble bursting	Market place jitters— see Table 3.
Subsidies ending?	LLM costs to rise 10x–100x ?

2 Gap Analysis. SE: What is Needed?

Recent work has shown that AI has shifted much developer time toward code review [68] and reading code versus writing code [40]. This often requires a complex, error-prone process of comparative comprehension [60, 61], where developers must understand code and how a change alters behavior. Hence we say

Trend 1: *Our graduates need a deeper understanding of code and how it can be improved.*

Also, Developers should not be content with writing acceptance tests from the requirements they provide to AI tools; they should have well developed plans to test different properties and test them at different levels in the code. For example, “Vibe Coding”—coding via natural language prompts [39]—is seeing serious industry adoption [8] with many available tools [88]. Results can be, initially, impressive with developers delivery systems much faster. But LLMs are probabilistic with limited world knowledge and will make mistakes (Table 1). As developers cede governance, such mistakes grow harder to find, fix, and can have grave consequences. As Cerf warns [7]: “In high-risk applications ... much more scrutiny might be advisable.”

Trend 2: *Our graduates must better understand that testing is imperative when using AI tools.*

Another recent trend is marrying subsymbolic connectionist methods (LLMs) with symbolic methods [4, 30]. Subsymbolics are good for reasoning across large, heterogeneous, ill-structured

Subject	Context	Level	Description	Refs
Ugrad/grad:				
Kathryn Stolee	SW Construction	ugrad	300	LLMs for code; must include prompts/sessions in comments.
	Product Mgmt	ugrad/grad	400/500	Understanding user needs and pain points around using AI. [10]
Tim Menzies	SE+AI	ugrad/grad	491/591	Tactics AI optimization; for prompting documentation and code review.
John-Paul Ore	DevOps/ClassOps	classops	400/500	GitHub scripts for repo/permission management.
Sandeep Kuttal	Human-Computer Int.	ugrad/grad	400/500	Focus on interactions with software and hardware.
	Human-AI Int.	Res/Teach	400	Design and evaluation of AI-centric systems.
Wesley Assunção	SW Architecture	Teaching	400/500	LLMs to draft architectures, support design decisions, and fix issues. [54]
Marcelo d'Amorim	SW testing	teaching	404/504	Hands-on intro. to SW testing. Approach: (1) Prototype; (2) Test; and (3) Repeat.
Grad:				
Tim Menzies	Grad intro to se	grad	500	RAG over 10k pages to eval design novelty/requirements. [14, 77, 89]
Marcelo d'Amorim	Grad intro to testing	grad	712	Latest techniques to use AI for testing and to test AI-enabled systems.
Bowen Xu	Generative AI for SE	grad	591/791	Latest practice of AI usage in software development and benchmarking LLMs from both functional (such as accuracy) and non-functional (such as security, privacy, and robustness) perspectives. [2, 11–13, 17–22, 31–34, 42, 48, 49, 65, 75, 78, 79, 81, 83, 84, 91, 98, 103, 104, 106, 108, 110, 111, 113, 115, 116]
Research:				
Kathryn Stolee & Bowen Xu	Empirical SE	research		LLM impact on coding activities and emotions.
John-Paul Ore	Robotics SE	research		LVA models for robot arms; fine-tuning via demos.
Tim Menzies	SW Testing	research	-	Agentic systems to build better test inputs. [14]
	Search-Based SE	research	-	Using Learn better optimizer inputs [77, 89]
Marcelo d'Amorim	SW Testing	research	-	How to make software more reliable? My goal is to make software testing more trustworthy and scalable. I use testing in AI (e.g., to check safety of autonomous vehicles and find bugs in deep learning libraries) and AI for testing (e.g., using LLM to create test drivers and mine metamorphic properties). [52, 63, 64]
	SW Maintenance	research	-	Detection of quality issues, code refactoring, and vulnerability repair. [3, 41, 51, 69]
Wesley Assunção	SW Evolution	research	-	Software modernization and Code translation with LLMs. [6, 76, 87, 93]
Sandeep Kuttal	Human-Centered SE	research	-	Human behavior-driven algorithm and tool design [23–25, 43–46, 56, 71–73, 92]
Bowen Xu	Trustworthy AI for SE	research	-	How AI properties (such as safety, security, privacy, robustness, efficiency) impact software quality? [2, 11–13, 20–22, 26–29, 31–36, 42, 47–49, 58, 65, 67, 74, 75, 78–84, 86, 91, 95–106, 108–116]

Table 2: Current use of AI by CSC SE faculty in teaching and research.

sources. Symbolic methods, on the other hand, support causal reasoning, temporal trend analysis [85], and explanation (particularly required in high-stakes domains like medicine, law, or material science) for system acceptance criteria. Here, LLMs may not be the model *per se* but a way to query or update another model; alternatively, symbolic assertions may include fragments of vectors from an embedded space (generated by an LLM).

Trend 3: *Our graduates need to understand both symbolic and subsymbolic systems.*

As of this writing, a third important trend is recent fluctuations in the commercial AI market (see Table 3). Quite possibly, in the very near future, the cost of using LLMs will increase 10x to 100x. However, recent research has shown that using small open-weights models that are “focused” on a given project repository can even outperform expensive closed-weights models on complex software engineering tasks [70].

Trend 4: *Our graduates need to understand when/how/which LLM services can be replaced with something cheaper.*

A substantial portion of the world’s critical software infrastructure runs on legacy systems—often decades old, tightly coupled, sparsely documented, and deeply embedded in operational, financial, healthcare, transportation, and government contexts. These systems are not greenfield playgrounds for “vibe coding”; they are revenue-generating, safety-relevant assets with strict uptime, compliance, and backward-compatibility requirements. Introducing AI tools into such environments must therefore be done with

extreme care. While LLMs can assist with code translation, modernization, documentation, and vulnerability repair, naively accepting AI-generated refactorings or migrations risks subtle regressions, architectural drift, and violations of implicit invariants accumulated over years of evolution. In legacy contexts, the cost of breaking a working system far exceeds the benefit of rapid code generation. Our graduates must therefore learn how to apply AI under change-management discipline: using regression test suites, traceability links, staged rollouts, and architectural guardrails to ensure that modernization efforts preserve behavioral correctness and operational stability. [WA: I need to add references]

Trend 5: *Our graduates need to learn how to integrate AI tools in existing systems.*

3 Still, SE needs Engineering

Despite our radical enthusiasm for generative AI, together with our awareness of its drawbacks, we argue that commercial software demands a transition from “AI hacking” back to managed rigor. We define SE as follows (and underline terms are discussed, below):

Definition 1: Software Engineering

SE is a process where individuals or teams manage the creation and maintenance of products of required quality to meet stakeholders need, given the current constraints.

The contrast in Figure 1 is instructive. SE (Figure 1.a) should maintain feedback loops between planning and testing:

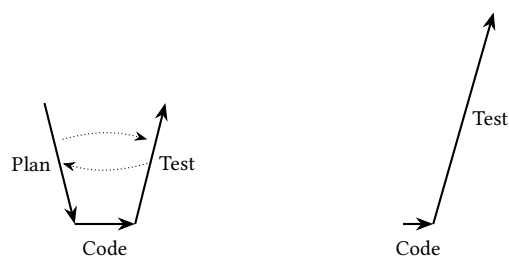


Fig 1.a. Current SE: Coding to gain feedback balancing planning and testing.

Fig 1.b. Vibe coding does little (or no) planning and is testing-intensive. Can generate overly elaborate code that is hard to maintain.

Figure 1: SE should have feedback loops between planning and testing But in vibe coding, there is little planning which leads to testing-intensive development.

- Plans generate expectations that must be tested;
- Testing generates feedback on which parts of the plan should be extended or restricted.

Vibe coding (Figure 1.b) abandons this discipline—leaping into code with no planning and no clear test plan. Paradoxically, this lack of initial planning leads to *increased* overall testing, since bugs in the vibed code are uncovered haphazardly after delivery and during operation.

While LLM tools provide useful starting points, programmers report difficulties understanding, editing, and debugging auto-generated code snippets, which significantly hinders task-solving effectiveness [16, 94]. Worse, even after generating code from an LLM, programmers often rewrite it to meet specific functional and non-functional requirements [50]. This is particularly concerning given that testing traditionally consumes nearly half of development time [5, 53]. Without managed planning, we simply defer the cost to an even more expensive testing phase, allowing bugs to happen in production, which can be even more costly.

3.1 Management and Maintenance.

Engineering requires that expectations between teams and users be carefully **managed** to ensure products evolve predictably. Prof. Kathryn Stolee and Sandeep Kuttal lead our efforts here: Stolee focuses on the product management and how to discover and assess user needs [10], while Kuttal explores the nuances of human-computer collaboration. Together, they ensure that SE remains disciplined rather than ad-hoc. For **maintenance**, Prof. Wesley Assunção’s work on software modernization and code quality improvement demonstrates that AI must manage technical debt without breaking architectural integrity [3, 41, 51, 69], while Prof. John-Paul Ore’s work with drones and DevOps scripts ensures that the maintenance of physical and virtual infrastructure remains rigorous.

3.2 Required Quality.

Quality is not a “vibe”; it is the result of rigorous assessment and verification [cite: 72]. Prof. Marcelo d’Amorim use AI to scale testing and to make it more trustworthy. For example, he has been using AI to synthesize test drivers and metamorphic testing to find bugs in language processors and deep learning libraries [52, 63, 64]. Prof. Wesley Assunção uses AI to repair vulnerabilities [51]. Prof. Bowen Xu, our expert in building LLMs, pushes this further by developing specialized security detectors and LLM-driven construction tools that ensure code meets high-stakes safety standards [cite: 13, 50, 51]. Without these quality gates, we lose the ability to know when a system is safe for the high-stakes deployment environments studied by Prof. John-Paul Ore [cite: 14, 73]. Quality concerns necessitate interaction with requirements since symbolic models allow us to discuss the “why” behind an agent’s choice.

3.3 Current Constraints and Optimization.

All projects must be shoe-horned into available resources. Such cost-effective optimizations will become important in the very near future, once LLM-subsidies end and we are forced to pay full cost for each query (see Table 3) and the full cost of using LLMs is moved to the user. Prof. Tim Menzies’ work focuses on **optimization**—using AI to navigate high-dimensional configuration spaces and hyperparameter exploration where both humans and algorithms are overwhelmed by too many choices. Optimization navigates trade-offs to find the best configurations under strict **constraints**.

3.4 The Neurosymbolic Shift.

Principle 2: What to teach (1):

Agentic systems are the future! But not all agents will be LLMs.

Combining all the above, we say we should move toward agentic systems that marry subsymbolic (i.e., LLM) power with symbolic (e.g., constraint-based reasoning) rigor. Subsymbolic methods are good for reasoning across large, heterogeneous, ill-structured sources. Symbolic methods, on the other hand, are useful for causal reasoning, temporal trend analysis, and explanatory purposes. These are particularly required for high-stakes domains (like medicine, law, or materials science) for the acceptance criteria of a system.

As a recent example of neurosymbolic synthesis in SE, we offer Shi et al.’s recent work from Queens University that combines LLMs with knowledge graphs to perform the complex temporal and causal reasoning required to isolate the bad code segments within a buggy commit [85]. In that work, LLMs are not the model but a method for querying a symbolic model.

Principle 3: What to teach (2):

NCState students should learn symbolic and subsymbolic (LLM) AI

To bridge our vision of Software Engineering with the formal requirements of computer science education, we must examine how these “radical” ideas align with established institutional and

Maturation through Correction: History shows that SE productivity is optimized after speculative crashes. Structured Programming (1970s), Object-Orientation (early 90s), and the Internet (2001 dot-com bust) each emerged from a “Trough of Disillusionment” to become foundational engineering pillars in a “Plateau of Productivity”. In this context, the January 2026 AI Market crash is not a failure of technology, but the necessary pivot toward engineering rigor. To summarize that crash....

The \$400B Reality Check: In late January 2026, Microsoft lost \$400B in one week—SE history’s largest single devaluation. MS Copilot costs hundreds per user [1], ChatGPT conversion remains 2–8% [57], 95% of enterprise apps fail to return revenue [62]. Trends in 2026 suggest market capital is rotating away from tech to Energy/Materials (up 10% since Jan 2026) since tools like DeepSeek make investors doubt that all AI will necessarily need expensive data centers.

The Subsidy Cliff: This crash, and the ones that might soon follow, tells us that the costs of use AI are about to radically change. Current AI API costs (\$50–\$200 per student) are venture capital -subsidized. But recent market trends suggest investors now doubt the value of the massive AI Capital expenditure (MS \$150B/yr; Amazon \$200B) against an \$800B revenue shortfall by 2030 [59]. Meanwhile the reported AI productivity gains remain modest ($\pm 20\%$) and may degrade quality [15].

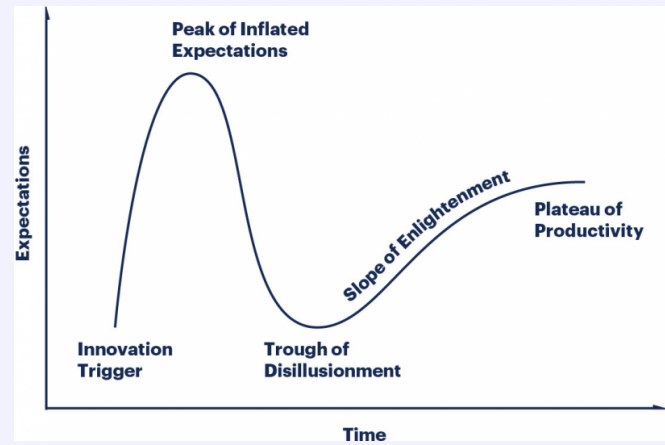


Table 3: Any plan to teach AI must prepare a post-subsidy reality. The AI "honeymoon" of cheap, VC-subsidized models may be about to end. Microsoft’s recent \$400B market correction signals a shift from AI exploration to the mandatory rigor of SE economics and analytics.

theoretical frameworks. The transition from "vibe coding" to engineered systems is not just an opinion; it is a pedagogical necessity mandated by the evolving standards of the field.

4 The Pedagogical Bridge: Theory and Standards

All the above strategies represent our collective vision, but they must also sit within the institutional and theoretical requirements of a top-tier American engineering program. To understand our connections to those institutional requirements, we now turn to three pillars: (i) **Bloom’s Taxonomy**, (ii) **ACM/IEEE Curricula**, and (iii) the **ABET** accreditation body. Each pillar is discussed below, together with our SE + AI vision.

4.1 Bloom Taxonomy

As shown in Figure 2, **Bloom’s Taxonomy** is a hierarchical model used to classify educational learning objectives into levels of complexity. It moves from basic recall (*Remembering*) to the highest form of cognitive processing (*Creating*).

Any transition to some new vision of AI teaching needs to touch on all aspects of Bloom (see Figure 3); i.e., it needs to address the ability to evaluate and ensure quality in systems that increasingly write themselves. This aligns with our definition of SE: *managed creation and maintenance of required quality given current constraints*.

4.2 ACM Curriculum

The **ACM/IEEE Computer Science Curricula** provides the global benchmark for what constitutes a “good” CS degree. In the following, we contrast the **CC2005** guidelines (which focused on knowledge-based programming) to the new **CS2023** standards (which prioritize competency-based evaluation of AI outputs). This

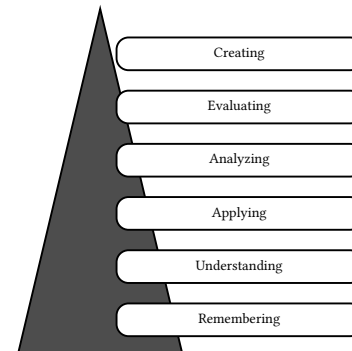


Figure 2: The Bloom Hierarchy of educational objectives. In the AI era, SE education must move from the base (Remembering/Applying) to the peak (Evaluating/Creating via synthesis) .

transition amplifies the *AI Vertigo* described earlier: the field risks adopting generative tools without the engineering discipline required for **managed** creation and **required quality** under current **constraints**.

The following areas show strong alignment between our current curriculum efforts and the CS2023 headers:

- **AI Overhaul (Pervasive):** AI woven throughout lifecycle (e.g., CSC 326_AI proposal).
- **Gen-AI Literacy:** We explore “Vibe Coding” and prompt engineering for RE and code review. AI is no longer a separate module but the substrate combined with proven engineering practices (Table 2). We avoid standalone “AI-tool” subjects for this reason.

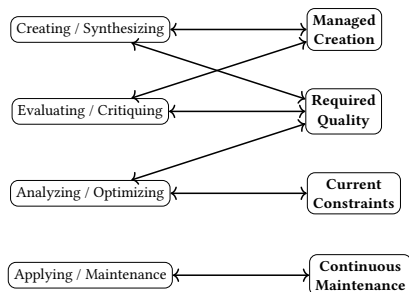


Figure 3: Mapping Bloom’s Higher-Order Thinking to the core components of our Software Engineering definition. High-level synthesis and critique are required to achieve “Managed Quality.”

- **Reliability:** Faculty research (testing, failure modes, vulnerability repair) aligns with ACM curriculum telemetry requirements.

That said, our brainstorming found four areas where the ACM CS2023 omit several critical “real-world” dimensions:

Challenge 1: ACM Curriculum: Four Missing Bits

Our curriculum must extend beyond standard ACM recommendations to address:

- (1) **Human-in-the-Loop Psychology (Management):** Combat decreased ownership from copy-paste workflows; teach scalable code review as **managed** collaboration.
- (2) **Economics & Sustainability (Constraints):** Bubble risk and AI energy costs are now technical **constraints**; we predict 5x–100x cost spikes (Table 3). *Solution: a “Token Bank” budget model.*
- (3) **Augmented Mental Models (Quality):** Students must retain “under-the-hood” basics to serve as effective oracles for AI logic, maintaining **required quality** and averting a maintenance disaster.
- (4) **Administrative Branding:** Transcripts must reflect AI competencies for job markets. *Solution: new courses carry explicit “AI Governance” titles.*

These gaps align with our stance of “**AI, carefully**”—radical enthusiasm tempered by critical reflection. Also, these **Four Missing Bits** map directly to our SE definition.

- **Management** ensures human oversight of AI workflows;
- **Constraints** demand economic and sustainability awareness;
- **Quality** requires students maintain symbolic reasoning to validate subsymbolic (LLM) outputs. This neurosymbolic synthesis—combining symbolic rigor with subsymbolic power—is essential for high-stakes systems. As stated in Principle 2: **NCState students should learn symbolic and subsymbolic (LLM) AI.** Without this integration, we risk graduating students who can prompt but cannot verify, violating our core commitment to **managed** creation of **required quality**.

4.3 ABET

ABET accreditation formalizes our “AI, carefully” stance into measurable Student Outcomes. The Accreditation Board for Engineering and Technology (ABET) is a global non-profit organization that certifies computing programs meet essential standards for quality, safety, and professional readiness. Every six years, the NC State Computer Science department undergoes comprehensive review by the Computing Accreditation Commission (CAC) to maintain accreditation [66].¹

The department maintains this connection through a cycle of formal accountability: detailed Self-Study Reports, on-site peer evaluations [9], and “Continuous Improvement” loops that collect student artifacts to prove competency achievement [66]. The question is whether AI-enhanced curricula maintain—or degrade—the engineering rigor ABET demands.

The integration of AI into the SE curriculum must be reconciled with the formal requirements of the ABET Computing Accreditation Commission (CAC). Based on current departmental self-study reports and faculty objectives, the curriculum must have the following outcomes:

- (1) **Complex Problem Solving:** Students must demonstrate the ability to analyze complex computing problems and apply principles of computing and other relevant disciplines to identify solutions [66].²
- (2) **Design and Evaluation:** Graduates must design, implement, and evaluate computing-based solutions that meet requirements within the context of the discipline [66].³
- (3) **Professional and Ethical Responsibility:** A core requirement is the ability to recognize professional responsibilities and make informed judgments based on legal and ethical principles [9].⁴
- (4) **Effective Communication and Teamwork:** The program must ensure students can communicate effectively in professional contexts and function as leaders or members of diverse teams [66].⁵
- (5) **Lifelong Learning and Trade-off Analysis:** Graduates are expected to engage in lifelong learning by exploring trade-offs between alternative solutions and innovating in response to change [9].⁶

The **Four Missing Bits** identified in our ACM analysis directly address ABET’s core competencies. We now show how each gap maps to specific Student Outcomes:

¹The next ABET on-site visit is scheduled for Fall 2028.

²ABET Student Outcome 1: “Analyze a complex computing problem and to apply principles of computing and other relevant disciplines to identify solutions.”

³ABET Student Outcome 2: “Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements.”

⁴ABET Student Outcome 4: “Recognize professional responsibilities and make informed judgments in computing practice based on legal and ethical principles.”

⁵ABET Student Outcomes 3 & 5 regarding communication and teamwork.

⁶NC State PEO 5: “Engage in lifelong learning by embracing change, exploring tradeoffs between alternative solutions, and innovating.”

Principle 4: ABET Alignment

Missing Bit 1: Human-in-the-Loop Psychology satisfies ABET Outcome 4 (“recognize professional responsibilities and make informed judgments”) [9]. Students who copy-paste AI outputs without ownership violate the “informed judgment” requirement. Teaching scalable code review ensures **managed** collaboration meets ethical standards.

Missing Bit 2: Economics & Sustainability satisfies ABET PEO 5 (“explore trade-offs between alternative solutions”) [9]. Understanding when to use expensive LLMs versus cheaper alternatives is exactly the trade-off analysis ABET requires. The predicted 5x-100x cost spike (Table 3) makes **constraints** awareness a professional competency, not an academic exercise.

Missing Bit 3: Augmented Mental Models satisfies ABET Outcome 2 (“design, implement, and evaluate computing-based solutions”) [66]. Students must retain “under-the-hood” basics to act as effective oracles for AI logic, enabling them to *evaluate* AI-generated code rather than blindly accept it. This maintains **required quality**.

Missing Bit 4: Administrative Branding satisfies ABET’s demand for demonstrable competencies. Transcripts must reflect AI-related skills to prove graduates meet industry standards for professional readiness.

While the transition to an AI-driven “competency-based” model (CS2023) is underway, ABET’s “Continuous Improvement” loop demands evidence that LLM interventions do not degrade student outcomes [66]. The risk of *automation bias*—where students inappropriately accept incorrect AI code—directly conflicts with Outcome 4’s requirement for “informed judgments” [9]. Similarly, students who cannot evaluate AI outputs violate Outcome 2’s mandate to “evaluate computing-based solutions” [66].

Without neurosymbolic synthesis, we violate Outcome 2 (evaluate) and Outcome 4 (informed judgment). Students need symbolic reasoning to validate subsymbolic (LLM) outputs. As stated in our core definition: SE requires **managed** creation of **required quality**. ABET formalizes this as professional competency. We risk graduating students who can prompt but cannot verify, failing both our educational mission and our accreditation obligations.

5 Software Engineering Remains Indispensable

While AI-assisted development can accelerate implementation and lower the barrier to prototyping, SE remains an indispensable discipline in the construction of *complex software systems*. More in detail:

- Requirements still need to be elicited, negotiated, prioritized, and documented. Stakeholders often have conflicting goals, implicit assumptions, and evolving constraints. Turning these into precise, testable, and traceable requirements remains a fundamentally human-centered engineering activity even in a scenario of growing “Vibe Coding”.
- Architectural decisions must be made explicitly. In a vibe coding scenario, systems often grow incrementally through

locally optimal decisions driven by immediate functionality. However, architecture concerns global structure: decomposition into components, definition of interfaces, allocation of responsibilities, data flow, and technology selection. These decisions determine scalability, deployability, evolvability, and integration capacity. Generative AI tools can suggest patterns or produce code fragments, but they do not assume responsibility for long-term structural coherence. Without deliberate architectural design, systems risk becoming tightly coupled, brittle, and expensive to evolve. Architecture provides the blueprint that constrains and guides code generation rather than merely reacting to it.

- Testing remains indispensable, particularly in an era of rapid, AI-assisted code generation. Vibe coding may accelerate feature development, but increased velocity amplifies the risk of subtle defects, integration faults, and unintended behaviors. Systematic testing practices are necessary to establish confidence in correctness and robustness. Testing is not merely about checking functionality; it is a mechanism for exposing hidden assumptions, validating edge cases, and evaluating behavior under realistic and adversarial conditions. Generative AI can help create tests, but determining what should be tested, defining meaningful oracles, ensuring adequate coverage, and interpreting failures require engineering judgment and domain understanding.
- Maintenance and evolution become even more critical in a vibe coding context. Rapid, AI-assisted code generation can lead to codebases that grow quickly but lack conceptual integrity, consistent abstractions, and clear documentation. When features are added through successive prompts and local refinements, design intent may remain implicit or fragmented across iterations. Over time, this can increase technical debt, obscure system invariants, and make future modifications riskier and more costly. Disciplined engineering practices (e.g., refactoring, version control, and continuous integration) are essential to preserve coherence.
- Accountability, ethics, and governance require human oversight. When systems fail, cause harm, or exhibit bias, responsibility cannot be attributed to a tool. Engineering practices provide traceability, auditability, and defensible decision-making processes.
- Safety-critical and mission-critical systems must be carefully designed, verified, and validated. Domains such as healthcare, transportation, finance, and aerospace require rigorous testing strategies, formal analysis when appropriate, and compliance with standards.

In short, Generative AI may change how code is produced, but it does not remove the need for disciplined engineering. Software Engineering remains indispensable because it provides the principles, methods, and accountability mechanisms necessary to transform ideas into reliable, safe, and trustworthy systems.

6 Summary: Bridging the Capability Gap

Based on the above, we could not propose the addition of any number of new subjects to our curriculum. That said, after extensive faculty brainstorming and prioritization, we have identified two

key areas that require entirely new subjects to address the current shortcomings:

- A **management-level subject** focused on the **economic cost-benefit perspectives** of AI adoption; and
- A **software analytics subject** to provide the **empirical monitoring** required to understand the true impact of these tools on project health.

Our analysis reveals a deliberate disconnect between current faculty strengths and the requirements of the AI era. While our current team aggressively uses AI as a substrate for research (see Table 2), the "Four Missing Bits" represent a vacuum that cannot be filled by existing personnel. To inject the required level of engineering rigor to SE, we must address two critical infrastructure needs:

- (1) **Human Capital:** We require **one to two new Software Engineering faculty members** specialized in **AI Financial Governance** and **Operational Auditing**. While our current faculty excel at empirical research, they all have full teaching loads of large classes (e.g. csc510 with 400+ students per year). Hence we require additional expertise to teach the economics and real-time governance of these systems. This is essential to ensure student outcomes meet the "informed judgment" requirements of ABET Outcome 4 [9].
- (2) **Computational Infrastructure:** We must establish a centralized **NC State LLM Token Bank**. As the "Subsidy Cliff" approaches (see Table 3), we estimate costs will reach **\$50 to \$500 per student, per semester** to facilitate serious engineering work. This resource will also allow students to measure and optimize energy consumption, directly addressing our **Green SE** goals.

Without these investments, we risk graduating students who can merely "vibe" rather than govern [16, 94]. [**WA: would be nice if we can make something related to "Think and do"**]

7 Management

objectives
outcomes
summary
graded components

8 Empirical

objectives
outcomes
summary
graded components

9 vibe to verify

? course should be 300 – 400 level. if they write the code what to do.

are they "just" testing the code?
workflows for testing.
for esned user programming: making it more techniques, based
lifelive stuff, workfloas. what are they using ai for and why.
vibe to verify subject.
- non-computing majors taught be a non comptuers scientists

need a section of whye se is needed.

add to table1 Storey's distinction (from her February 2026 blog post) is clean and quotable: Technical debt lives in the code — bad design choices that make software harder to modify over time. The metaphor implies you fix it by refactoring the code. Cognitive debt lives in developers' minds — the erosion of shared understanding of why a system is designed the way it is and how it can safely be changed. Crucially, even if AI agents produce code that could be easy to understand, the humans involved may have simply lost the plot and may not understand what the program is supposed to do, how their intentions were implemented, or how to possibly change it. Margaret-Anne Storey She grounds this in Peter Naur's old idea that a program is a "theory" distributed across the minds of a team. Vibe coding destroys that shared theory faster than it creates it. Why this matters for your paper: your doc already cites cognitive debt (Table 1) but doesn't name Storey or make this distinction explicit. Since your "Augmented Mental Models" missing bit is essentially about preventing cognitive debt, naming Storey's distinction there would sharpen the argument considerably — and it directly supports your case that SE governance (not just code cleanup) is what's needed.

two endings: one reorg and one expansion.

References

- [1] Ai startups are bad businesses. <https://www.youtube.com/watch?v=OYlQyPo-L4g>, September 2025. Ref [1].
- [2] Saima Afrin, Bowen Xu, and Antonio Mastropaolo. Is quantization a deal-breaker? empirical insights from large code models. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 1–13. IEEE, 2025.
- [3] Nafisa Ahmed, Hin Chi Kwok, Mohammad Hamdaqa, and Wesley KG Assunção. Smatch-m-llm: Semantic similarity in metamodel matching with large language models. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, pages 199–210. IEEE, 2025.
- [4] Bikram Pratim Bhuyan, Amar Ramdane-Cherif, Ravi Tomar, and TP Singh. Neuro-symbolic artificial intelligence: a survey. *Neural Computing and Applications*, 36(21):12809–12844, 2024.
- [5] F. P. Brooks. *The Mythical Man-Month*. Addison-Wesley, Reading, MA, USA, 1975.
- [6] Luiz Carvalho, Thelma Elita Colanzi, Wesley KG Assunção, Alessandro Garcia, Juliana Alves Pereira, Marcos Kalinowski, Rafael Maiani de Mello, Maria Julia de Lima, and Carlos Lucena. On the usefulness of automatically generated microservice architectures. *IEEE Transactions on Software Engineering*, 50(3):651–667, 2024.
- [7] Vinton G. Cerf. Does ai now represent a paradigm shift? *Communications of the ACM*, 69(2):5–5, February 2026.
- [8] Cognizant Insights Team. The market has spoken: Vibe coding is serious business, July 2025.
- [9] CSC Faculty Meeting. Accreditation status update and program educational objectives (peo) review, October 2025. Accreditation Board for Engineering and Technology (ABET) update presentation.
- [10] Hannah Estes, Katie Hollowell, John Cheek, Shiv Patel, Hanna Reese, and Kathryn T. Stolee. Advancing software product management education: Insights from an industry survey. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion '25*, page 999–1009, New York, NY, USA, 2025. Association for Computing Machinery.
- [11] Sen Fang, Weiyuan Ding, Antonio Mastropaolo, and Bowen Xu. Smaller=weaker? benchmarking robustness of quantized llms in code generation. *arXiv preprint arXiv:2506.22776*, 2025.
- [12] Sen Fang, Weiyuan Ding, and Bowen Xu. Evaloop: A self-consistency-centered framework for assessing large language model robustness in programming. *arXiv preprint arXiv:2505.12185*, 2025.
- [13] Sen Fang, Weiyuan Ding, and Bowen Xu. Evaloop: Assessing llm robustness in programming from a self-consistency perspective. *arXiv preprint arXiv:2505.12185*, 2025.
- [14] Kishan Kumar Ganguly and Tim Menzies. From coverage to causes: Data-centric fuzzing for javascript engines. *arXiv preprint arXiv:2512.18102*, 2025.
- [15] GitClear. Coding on copilot: Data shows ai's downward pressure on code quality. 2024.

- [16] GitHub. Github copilot: Your ai pair programmer. <https://github.com/features/copilot>.
- [17] Chen Gong, Zhou Yang, Yunpeng Bai, Junda He, Jieke Shi, Kecen Li, Arunesh Sinha, Bowen Xu, Xinwen Hou, David Lo, et al. Baffle: Backdoor attack in offline reinforcement learning. *arXiv preprint arXiv:2210.04688*, 2023.
- [18] Chen Gong, Zhou Yang, Yunpeng Bai, Junda He, Jieke Shi, Kecen Li, Arunesh Sinha, Bowen Xu, Xinwen Hou, David Lo, et al. Baffle: Hiding backdoors in offline reinforcement learning datasets. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 2086–2104. IEEE, 2024.
- [19] Chen Gong, Zhou Yang, Yunpeng Bai, Junda He, Jieke Shi, Arunesh Sinha, Bowen Xu, Xinwen Hou, Guoliang Fan, and David Lo. Mind your data! hiding backdoors in offline reinforcement learning datasets. *arXiv preprint arXiv:2210.04688*, 3, 2022.
- [20] Md Nazmul Haque, Elizabeth Lin, Lawrence Arkoh, Biruk Tadesse, and Bowen Xu. Secure or suspect? investigating package hallucinations of shell command in original and quantized llms. *arXiv preprint arXiv:2512.08213*, 2025.
- [21] Md Nazmul Haque, Hua Yang, Zhou Yang, and Bowen Xu. How does quantization impact privacy risk on llms for code? In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*, pages 50–60. IEEE, 2025.
- [22] Md Nazmul Haque, Hua Yang, Zhou Yang, and Bowen Xu. How quantization impacts privacy risk on llms for code? *arXiv preprint arXiv:2508.00128*, 2025.
- [23] J. Hart, J. Aubuchon, and Sandeep Kaur Kuttal. Feasibility of using youtube conversations for pair programming intent classification. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2022.
- [24] Jacob Hart, Jake AuBuchon, Alexander McAuliffe, Shandler Mason, and Sandeep Kaur Kuttal. Navigating nlu challenges in pair programming agents: A study on data size, gender, language, & domain effects. In *Artificial Intelligence in HCI: 5th International Conference, AI-HCI 2024, Held as Part of the 26th HCI International Conference, HCII 2024, Proceedings, Part I*, pages 356–375, Washington, DC, USA, 2024. Springer-Verlag.
- [25] Jacob Hart, Sandeep Kaur Kuttal, and Marcus Ensley. Gender effects on creativity when pair programming with a human vs. an agent. In H. Degen and S. Ntoa, editors, *Artificial Intelligence in HCI*. Springer International Publishing, 2023.
- [26] Junda He, Bowen Xu, Zhou Yang, DongGyun Han, Chengran Yang, Jiakun Liu, Zhipeng Zhao, and David Lo. Ptm4tag+: Tag recommendation of stack overflow posts with pre-trained models. *Empirical Software Engineering*, 30(1):28, 2025.
- [27] Junda He, Bowen Xu, Zhou Yang, DongGyun Han, Chengran Yang, and David Lo. Ptm4tag: sharpening tag recommendation of stack overflow posts with pre-trained models. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, pages 1–11, 2022.
- [28] Junda He, Zhou Yang, Jieke Shi, Chengran Yang, Kisub Kim, Bowen Xu, Xin Zhou, and David Lo. Curiosity-driven testing for sequential decision-making process. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–14, 2024.
- [29] Junda He, Xin Zhou, Bowen Xu, Ting Zhang, Kisub Kim, Zhou Yang, Ferdian Thung, Ivana Clairine Irsan, and David Lo. Representation learning for stack overflow posts: How far are we? *ACM Transactions on Software Engineering and Methodology*, 33(3):1–24, 2024.
- [30] Pascal Hitzler and Md Kamruzzaman Sarker. Neuro-symbolic artificial intelligence: The state of the art. 2022.
- [31] Aftab Hussain, Md Rafiqul Islam Rabin, Toufique Ahmed, Mohammad Amin Alipour, and Bowen Xu. Occlusion-based detection of trojan-triggering inputs in large language models of code. *arXiv preprint arXiv:2312.04004*, 2023.
- [32] Aftab Hussain, Md Rafiqul Islam Rabin, Toufique Ahmed, Mohammad Amin Alipour, Bowen Xu, and Stephen Huang. Finding trojan triggers in code llms: An occlusion-based human-in-the-loop approach. In *2025 IEEE/ACM 4th International Conference on AI Engineering—Software Engineering for AI (CAIN)*, pages 281–282. IEEE, 2025.
- [33] Aftab Hussain, Md Rafiqul Islam Rabin, Toufique Ahmed, Navid Ayoobi, Bowen Xu, Prem Devanbu, and Mohammad Amin Alipour. A survey of trojans in neural models of source code: Taxonomy and techniques. *arXiv preprint arXiv:2305.03803*, 2023.
- [34] Aftab Hussain, Md Rafiqul Islam Rabin, Toufique Ahmed, Bowen Xu, Premkumar Devanbu, and Mohammad Amin Alipour. Trojans in large language models of code: A critical review through a trigger-based taxonomy. *arXiv preprint arXiv:2405.02828*, 2024.
- [35] Aftab Hussain, Md Rafiqul Islam Rabin, Bowen Xu, David Lo, and Mohammad Amin Alipour. A study of variable-role-based feature enrichment in neural models of code. In *2023 IEEE/ACM International Workshop on Interpretability and Robustness in Neural Software Engineering (InteNSE)*, pages 14–20. IEEE, 2023.
- [36] Aftab Hussain, Sadegh AlMahdi Kazemi Zarkouei, Md Rafiqul Islam Rabin, Mohammad Amin Alipour, Sen Lin, and Bowen Xu. Capturing the effects of quantization on trojans in code llms. *arXiv preprint arXiv:2505.14200*, 2025.
- [37] K. Jesse, T. Ahmed, P. Devanbu, and E. Morgan. Large language models and simple, stupid bugs. In *Proc. MSR*, pages 563–575, 2023.
- [38] S. Kabir, D. N. Udo-Imeh, B. Kou, and T. Zhang. Is stack overflow obsolete? an empirical study of chatgpt answers. In *Proc. CHI*, pages 1–17, 2024.
- [39] Andrej Karpathy. There’s a new kind of coding i call “vibe coding”. <https://x.com/karpathy/status/188619218480149383>, February 2025. Post on X (Twitter), Accessed: 2026-02-13.
- [40] Caitlin Sadowski Sebastian Elbaum Kathryn T. Stolee, Tobias Welp. 10 years later: revisiting how developers search for code. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025.
- [41] Hongyu Kuang, Ning Zhang, Hui Gao, Xin Zhou, Wesley KG Assuncao, Xiaoxing Ma, Dong Shao, Guoping Rong, and He Zhang. Brevity is the soul of wit: Condensing code changes to improve commit message generation. In *Proceedings of the 16th International Conference on Internetware*, pages 389–401, 2025.
- [42] Ummay Kulsum, Haotian Zhu, Bowen Xu, and Marcelo d’Amorim. A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*, pages 103–111, 2024.
- [43] S. Kuttal, J. Myers, S. Gurka, D. Magar, D. Piorkowski, and R. Bellamy. Towards designing conversational agents for pair programming: Accounting for creativity strategies and conversational styles. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2020.
- [44] Sandeep Kaur Kuttal, Jacob Hart, Marcus Ensley, and Shandler Mason. Pair programming in the lab vs. wild: A qualitative analysis of creativity strategies and dialogue styles for agent training data. In *Artificial Intelligence in HCI: 6th International Conference, AI-HCI 2025, Held as Part of the 27th HCI International Conference, HCII 2025, Gothenburg, Sweden*, 2025.
- [45] Sandeep Kaur Kuttal, B. Ong, K. Kwasny, and P. Robe. Trade-offs for substituting a human with an agent in a pair programming context: The good, the bad and the ugly. In *Proceedings of the International Conference on Human Factors in Computing Systems (CHI)*, 2021. Honorable Mention.
- [46] Sandeep Kaur Kuttal, A. Sedhain, and J. AuBuchon. Designing a gender-inclusive conversational agent for pair programming: An empirical investigation. In H. Degen and S. Ntoa, editors, *Artificial Intelligence in HCI*, pages 59–75. Springer International Publishing, 2021.
- [47] Xiang Lan, Tim Menzies, and Bowen Xu. Smart cuts: Enhance active learning for vulnerability detection by pruning bad seeds. *arXiv preprint arXiv:2506.20444*, 2025.
- [48] Quoc Hung Le, Thanh Le-Cong, Bach Le, and Bowen Xu. Signature in code backdoor detection, how far are we? *arXiv preprint arXiv:2510.13992*, 2025.
- [49] Ze Shi Li, Nowshin Nawar Arony, Ahmed Musa Awon, Daniela Damian, and Bowen Xu. Ai tool use and adoption in software development by individuals and organizations: a grounded theory study. *arXiv preprint arXiv:2406.17325*, 2024.
- [50] J. T. Liang, C. Yang, and B. A. Myers. Understanding the usability of ai programming assistants, 2023.
- [51] Kevin Lira, Balduino Fonseca, Wesley K. G. Assuncao, Davy Baya, and Marcio Ribeiro. Beyond code explanations: a ray of hope for cross-language vulnerability repair. In *2nd IEEE/ACM International Conference on AI-powered Software (AIware)*, pages 01–09, 2025.
- [52] Lingyun Liu, Feiran Qin, Owolabi Legunsen, and Marcelo d’Amorim. Bug histories as sources of compiler fuzzing mutators, 2025.
- [53] Daniel Long, Scott Drylie, Jonathan D. Ritschel, and Clay Koschnick. An assessment of rules of thumb for software phase management, and the relationship between phase effort and schedule success. *IEEE Transactions on Software Engineering*, 50(2):209–219, 2024.
- [54] Atif Mashkoor, Wesley KG Assuncao, and Alexander Egyed. Teaching engineering of ai-intensive systems. *IEEE Software*, 41(2):30–35, 2023.
- [55] A. Mastropaolo, M. Di Penta, and G. Bavota. Towards automatically addressing self-admitted technical debt: How far are we? In *Proc. ASE*, pages 585–597, 2023.
- [56] A. McAuliffe, J. Hart, and Sandeep Kaur Kuttal. Evaluating gender effects in pair programming conversations. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2022.
- [57] McKinsey. Seizing the agentic-ai advantage. Technical report, McKinsey & Company, 2025.
- [58] Deep Mehta, Kartik Rawool, Subodh Gujar, and Bowen Xu. Automated devops pipeline generation for code repositories using large language models. *arXiv preprint arXiv:2312.13225*, 2023.
- [59] METR. Early 2025 ai experienced os dev study, July 2025. Ref [6].
- [60] Justin Middleton, John-Paul Ore, and Kathryn T Stolee. Barriers for students during code change comprehension. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE ’24, New York, NY, USA*, 2024. Association for Computing Machinery.
- [61] Justin Middleton and Kathryn T. Stolee. Understanding similar code through comparative comprehension. In *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 1–11, 2022.
- [62] MIT NANDA. State of ai in business 2025 report, July 2025. Ref [3].
- [63] M M Abid Naziri, Shinhae Kim, Feiran Qin, Marcelo d’Amorim, and Saikat Dutta. Testing deep learning libraries via neurosymbolic constraint learning,

- 2026.
- [64] M M Abid Naziri, Stefano Carlo Lambertenghi, Andrea Stocco, and Marcelo d’Amorim. Misbehavior forecasting for focused autonomous driving systems testing, 2025.
- [65] Md Nazmul Haque, Hua Yang, Zhou Yang, and Bowen Xu. How quantization impacts privacy risk on llms for code? *arXiv e-prints*, pages arXiv–2508, 2025.
- [66] NC State Department of Computer Science. Abet self-study report for the department of computer science at north carolina state university. Technical report, NC State University, 2022. Final Self-Study Report for the Computing Accreditation Commission (CAC).
- [67] Truong Giang Nguyen, Thanh Le-Cong, Hong Jin Kang, Ratnadira Widayarsi, Chengran Yang, Zhipeng Zhao, Bowen Xu, Jiayuan Zhou, Xin Xia, Ahmed E Hassan, et al. Multi-granularity detector for vulnerability fixes. *IEEE Transactions on Software Engineering*, 49(8):4035–4057, 2023.
- [68] Kathryn T. Stolee Nick Case, John-Paul Ore. Understanding comparative comprehension barriers for students during code review through simplification. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion ’25)*, 2025.
- [69] Feifei Niu, Junqian Shao, Christoph Mayr-Dorn, Liguo Huang, K.G. Wesley Assunção, Chuanyi Li, Jidong Ge, and Alexander Egyed. Refactoring ≠ bug-inducing: Improving defect prediction with code change tactics analysis. In *IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, pages 527–538, 2025.
- [70] Jinjun Peng, Magnus Saebø, Tianjun Zhong, Yi-Jie Cheng, Junfeng Yang, Baishakhi Ray, Simin Chen, and Yangruibo Ding. Swe-spot: Building small repo-experts with repository-centric learning, 2026.
- [71] P. Robe, S. Kuttal, Y. Zhang, and R. Bellamy. Can machine learning facilitate remote pair programming? challenges, insights and implications. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2020.
- [72] P. Robe, Sandeep Kaur Kuttal, J. Aubuchon, and J. Hart. Pair programming conversations with agents vs. developers: Challenges & opportunities for se community. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.
- [73] Prerana Robe and Sandeep Kuttal. Designing pairbuddy: A conversational agent for pair programming. *ACM Transactions on Computer-Human Interaction*, 29(4), 2021. Journal-first paper; Invited talk at CHI 2023.
- [74] Mohammad Sadegh Sheikhaei, Yuan Tian, Shaowei Wang, and Bowen Xu. An empirical study on the effectiveness of large language models for satd identification and classification. *arXiv e-prints*, pages arXiv–2405, 2024.
- [75] Mohammad Sadegh Sheikhaei, Yuan Tian, Shaowei Wang, and Bowen Xu. Understanding the effectiveness of llms in automated self-admitted technical debt repayment. *arXiv e-prints*, pages arXiv–2501, 2025.
- [76] Alfred Santa Molison, Marcia Moraes, Glaucia Melo, Fabio Santos, and Wesley KG Assuncao. Is llm-generated code more maintainable & reliable than human-written code? In *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 151–162. IEEE, 2025.
- [77] Lohith Senthikumar and Tim Menzies. Can large language models improve se active learning via warm-starts? *ACM Transactions on Software Engineering and Methodology*, 2026.
- [78] Mohammad Sadegh Sheikhaei, Yuan Tian, Shaowei Wang, and Bowen Xu. An empirical study on the effectiveness of large language models for satd identification and classification. *Empirical Software Engineering*, 29(6):159, 2024.
- [79] Jieke Shi, Zhou Yang, Junda He, Bowen Xu, Dongsun Kim, DongGyun Han, and David Lo. Finding safety violations of ai-enabled control systems through the lens of synthesized proxy programs. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [80] Jieke Shi, Zhou Yang, Junda He, Bowen Xu, and David Lo. Can identifier splitting improve open-vocabulary language model of code? In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 1134–1138. IEEE, 2022.
- [81] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. Smaller, faster, greener: Compressing pre-trained code models via surrogate-assisted optimization. *arXiv preprint arXiv:2309.04076*, 2023.
- [82] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. Towards smaller. *Faster, and Greener Language Models of Code*. *arXiv e-prints*, 2023.
- [83] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. Greening large language models of code. In *Proceedings of the 46th international conference on software engineering: software engineering in society*, pages 142–153, 2024.
- [84] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. Compressing pre-trained models of code into 3 mb. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–12, 2022.
- [85] Yu Shi, Hao Li, Bram Adams, and Ahmed E. Hassan. Beyond blame: Rethinking szz with knowledge graph search, 2026.
- [86] Yucen Shi, Ying Yin, Zhengkui Wang, David Lo, Tao Zhang, Xin Xia, Yuhai Zhao, and Bowen Xu. How to better utilize code graphs in semantic code search? In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 722–733, 2022.
- [87] Vanshika Singh, Caglar Korlu, and Wesley KG Assunção. Experiences on using large language models to re-engineer a legacy system at volvo group. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 102–112. IEEE, 2025.
- [88] Matthew S. Smith. The best ai coding tools you can use right now. *IEEE Spectrum*, June 2025. Accessed: 2026-02-13.
- [89] Srinath Srinivasan and Tim Menzies. Beyond the prompt: Assessing domain knowledge strategies for high-dimensional llm optimization in software engineering. *arXiv preprint arXiv:2602.02752*, 2026.
- [90] Margaret-Anne Storey. How generative and agentic ai shift concern from technical debt to cognitive debt, February 2026. <https://margaretstorey.com/blog/2026/02/09/cognitive-debt/>.
- [91] Zhenyu Sun, Haotian Zhu, Bowen Xu, Xiaoning Du, Li Li, and David Lo. Llm as runtime error handler: A promising pathway to adaptive self-healing of software systems. *arXiv preprint arXiv:2408.01055*, 2024.
- [92] Benyamin Tabarsi, Heidi Reichert, Ally Limke, Sandeep Kuttal, and Tiffany Barnes. Llms’ reshaping of people, processes, products, and society in software development: A comprehensive exploration with early adopters. *arXiv*, 2025. *arXiv preprint arXiv:2503.05012*.
- [93] Biruk Tadesse, Vikram Nitin, Mazin Salah, Baishakhi Ray, Marcelo d’Amorim, and Wesley Assunção. Code quality analysis of translations from c to rust, 2026.
- [94] P. Vaithilingam, T. Zhang, and E. L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by llms. In *Proc. CHI EA*, pages 1–7, 2022.
- [95] Yanan Wu, Zhou Yang, Bowen Xu, Bo Wang, Bach Le, and David Lo. Learning from the best: What makes popular hugging face models? a registered report. In *This paper has been accepted at the International Conference on Software Maintenance and Evolution (ICSME 2025) Registered Report Track*, 2025.
- [96] Bowen Xu. Learning to interpret knowledge from software q&a sites. 2021.
- [97] Bowen Xu, Thong Hoang, Abhishek Sharma, Chengran Yang, Xin Xia, and David Lo. Post2vec: Learning distributed representations of stack overflow posts. *IEEE Transactions on Software Engineering*, 48(9):3423–3441, 2021.
- [98] Bowen Xu, Thanh-Dat Nguyen, Thanh Le-Cong, Thong Hoang, Jiakun Liu, Kisub Kim, Chen Gong, Changan Niu, Chenyu Wang, Bach Le, et al. Are we ready to embrace generative ai for software q&a? In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1713–1717. IEEE, 2023.
- [99] Chengran Yang, Jiakun Liu, Bowen Xu, Christoph Treude, Yunbo Lyu, Junda He, Ming Li, and David Lo. Apidocbooster: An extract-then-abstract framework for augmenting api documentation. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 36–47. IEEE, 2025.
- [100] Chengran Yang, Bowen Xu, Junaed Younus Khan, Gias Uddin, Donggyun Han, Zhou Yang, and David Lo. Aspect-based api review classification: How far can pre-trained transformer model go? In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 385–395. IEEE, 2022.
- [101] Chengran Yang, Bowen Xu, Jiakun Liu, and David Lo. Techsumbot: A stack overflow answer summarization tool for technical query. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 132–135. IEEE, 2023.
- [102] Chengran Yang, Bowen Xu, Ferdian Thung, Yucen Shi, Ting Zhang, Zhou Yang, Xin Zhou, Jieke Shi, Junda He, Donggyun Han, et al. Answer summarization for technical queries: Benchmark and new approach. In *PROCEEDINGS OF THE 37th IEEE/ACM international conference on automated software engineering*, pages 1–13, 2022.
- [103] Hua Yang, Alejandro Velasco, Sen Fang, Bowen Xu, and Denys Poshyvanyk. Understanding privacy risks in code models through training dynamics: A causal approach. *arXiv preprint arXiv:2512.07814*, 2025.
- [104] Hua Yang, Alejandro Velasco, Thanh Le-Cong, Md Nazmul Haque, Bowen Xu, and Denys Poshyvanyk. How do semantically equivalent code transformations impact membership inference on llms for code? *arXiv preprint arXiv:2512.15468*, 2025.
- [105] Zhou Yang, Jieke Shi, Muhammad Hilmi Asyrof, Bowen Xu, Xin Zhou, DongGyun Han, and David Lo. Prioritizing speech test cases. *ACM Transactions on Software Engineering and Methodology*, 34(4):1–27, 2025.
- [106] Zhou Yang, Bowen Xu, Jie M Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. Stealthy backdoor attack for code models. *IEEE Transactions on Software Engineering*, 50(4):721–741, 2024.
- [107] B. Yetistiren, I. Ozsoy, and E. Tuzun. Assessing the quality of github copilot’s code generation. In *Proc. PROMISE*, pages 62–71, 2022.
- [108] Ting Zhang, DongGyun Han, Venkatesh Vinayakara, Ivana Clairine Irsan, Bowen Xu, Ferdian Thung, David Lo, and Lingxiao Jiang. Duplicate bug report detection: How far are we? *ACM Transactions on Software Engineering and*

- Methodology, 32(4):1–32, 2023.
- [109] Ting Zhang, Bowen Xu, Ferdian Thung, Stefanus Agus Haryono, David Lo, and Lingxiao Jiang. Sentiment analysis for software engineering: How far can pre-trained transformer models go? In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 70–80. IEEE, 2020.
- [110] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, Junda He, and David Lo. Generation-based code review automation: How far are we? In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*, pages 215–226. IEEE, 2023.
- [111] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. Multi-llm collaboration+ data-centric innovation= 2x better vulnerability repair. *arXiv preprint arXiv:2401.15459*, 2024.
- [112] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*, pages 1–13, 2024.
- [113] Xin Zhou, Kisub Kim, Bowen Xu, Jiakun Liu, DongGyun Han, and David Lo. The devil is in the tails: How long-tailed code distributions impact large language models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 40–52. IEEE, 2023.
- [114] Xin Zhou, Bowen Xu, DongGyun Han, Zhou Yang, Junda He, and David Lo. Ccbert: Self-supervised code change representation learning. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 182–193. IEEE, 2023.
- [115] Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Thanh Le-Cong, Junda He, Bach Le, and David Lo. Patchzero: Zero-shot automatic patch correctness assessment. *arXiv preprint arXiv:2303.00202*, 2023.
- [116] Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Hung Huu Nguyen, Thanh Le-Cong, Junda He, Bach Le, and David Lo. Leveraging large language model for automatic patch correctness assessment. *IEEE Transactions on Software Engineering*, 2024.